

Graphics and sound - the blitter and the host displays

Graphics and sound, done by the host

How the RISC OS Pi 4 emulator puts a desktop in a window on Windows and on the Mac, moves the drawing RISC OS does in ARM code onto the host, and plays sound — all without emulating the GPU or any audio chip.

A Raspberry Pi 4's graphics and sound belong to its VideoCore GPU and Broadcom's closed firmware. The emulator models neither. It answers the few questions RISC OS asks its firmware, reads the screen straight out of guest memory, and draws it with the host's own GPU API. Sound is handled the same way: it is a conversation over a message queue, answered by QEMU's audio back end.

There is no GPU in QEMU — we are the GPU.

That line, from the fork's graphics design, is the thread through every chapter.

These documents

Chapter	What it covers
1. We are the GPU	what RISC OS asks for, and where the screen lives
2. The window owns the thread	the Windows front end, Direct3D 11
3. The same program, another idiom	the Mac front end, Metal, and Apple Events
4. The pointer is a sprite again	the hardware pointer, answered by the host

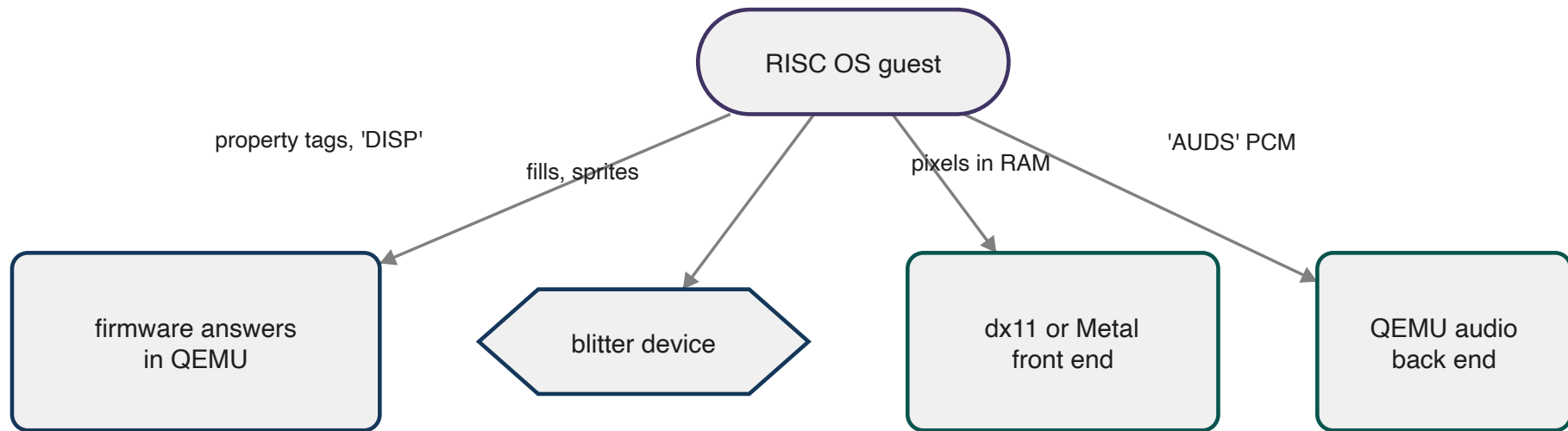
These documents, continued

Chapter	What it covers
5. Moving drawing to the host	why not the GPU; the blitter device
6. GVFill, line by line	the RISC OS module that feeds the blitter
7. Knowing what the guest drew	tearing, the damage word, screen banks
8. COMPLETE is the clock	sound over VCHIQ, clocked by the host
9. Measured, and open	the numbers, the risks, the drift

At a glance

Screen	guest RAM, mapped lock-free and decoded by a GPU shader every frame
Windows	<code>-display dx11</code> : Direct3D 11, C++17 behind a C boundary
Mac	<code>-display metal</code> : Metal, Objective-C, plus an Apple Events surface
Blitter	a new MMIO device plus GVFill, an assembly RISC OS module
Sprite plot	490 μ s in the guest, 80 μ s on the host: 6.1x , 0 pixels different
Sound	VCHIQ 'AUDS', clocked by the host sound card: 0.998x real time

Where each piece runs



These documents build on two others. The [QEMU A72 walkthrough](#) covers the machine: the mailbox and VCHIQ peers, the EDID answer, the timer thread and the DMA copies. The [fake it in software walkthrough](#) covers the design principle behind all of it.

1. We are the GPU

On a real Raspberry Pi, the ARM cores never touch the display hardware directly. RISC OS asks the VideoCore GPU for a framebuffer, draws into it, and the GPU scans it out to HDMI. The same GPU mixes sound and composites the hardware mouse pointer. All of it runs Broadcom's closed firmware.

The emulator runs none of that firmware. This chapter covers what it does instead: the questions RISC OS asks, the answers QEMU gives, and where the pixels actually live.

The principle

The fork's graphics design states the principle it follows in one paragraph:

The design principle, landed where it belongs: **there is no GPU in QEMU — we are the GPU**. Everything the ROM asks its GPU, the host answers: the property channel, 'AUDS', and now 'DISP'.

It is the graphics form of the project's broader rule: emulate the *functions* RISC OS needs, and emulate hardware only as a last resort. Nothing VideoCore-like exists in the fork. There is no GPU command stream, no 3D core and no HDMI encoder. There are answers.

What RISC OS asks its firmware

RISC OS's video and sound drivers talk to the VideoCore through two channels:

Channel	What it carries	Who answers in the fork
mailbox channel 8, property tags	modes, EDID, framebuffer allocation, palette	QEMU's property model, extended
mailbox channel 3, VCHIQ	named services: sound, the pointer, and others	the fork's VCHIQ peer

The property channel was already in QEMU; the fork extended it with an EDID answer and wide modes. VCHIQ was new. The [QEMU A72 walkthrough](#) tells how the peer was built to get past a wait with no timeout.

VCHIQ: refused, then answered

The VCHIQ peer began by refusing every service the guest opened. That was correct at the time: accepting 'AUDS' led into a second wait with no timeout. A refusal also leaves the video driver setting modes through the property channel, which QEMU already modelled. The graphics and sound work then turned two of those refusals into answers:

Service	What it is	Today
'AUDS'	the sound service: PCM out to the GPU	accepted: chapter 8
'DISP'	dispmanx: display elements, used for the hardware pointer	accepted: chapter 4
'UPDH'	update notifications, a companion to 'DISP'	opened; nothing is ever sent
'GCMD', 'TVSV'	GPU commands, TV service	still refused

The accept-or-refuse decision is a short branch on the service's four-character code, in `hw/misc/bcm2835_vchiq.c`. Anything it does not recognise gets a `CLOSE`, which the guest treats as a clean refusal before carrying on:

```
if (fourcc == VCHIQ_FOURCC_AUDS && !s->auds_open) {
    s->auds_open = true;
    s->auds_port = srcport;
    replies += vchiq_queue_msg_data(s,
        VCHIQ_MAKE_MSG(VCHIQ_MSG_OPENACK, VCHIQ_AUDS_VC_PORT, srcport),
        &ack, 4);
} else if (fourcc == VCHIQ_FOURCC_DISP && !s->disp_open) {
    ...
    /* the same, for the pointer */
} else {
    replies += vchiq_queue_msg(s,
        VCHIQ_MAKE_MSG(VCHIQ_MSG_CLOSE, 0, srcport), 0);
}
```

Accepting 'DISP' did not move mode setting off the property channel. The ROM's dispmanx initialisation runs after the framebuffer is allocated, so modes are still negotiated through property tags.

Where the screen lives

The framebuffer is **ordinary guest RAM**. QEMU's `bcm2835_fb` model reserves VideoCore memory, 64 MiB by default, at `vram_base`:

Address	Contents
<code>vram_base</code>	the palette: 256 words of <code>0x00BBGGRR</code> , written by the set-palette tag
<code>vram_base + 1 MiB</code>	the framebuffer itself, <code>pitch × rows</code> bytes

The video driver negotiates the mode through a batch of property tags: size, depth, pixel order, pitch, then an allocate. The property model gathers the tags into one configuration, committed once per mailbox buffer.

A seqlock around the configuration

A front end reads the configuration from its own thread without taking any QEMU lock. So the fork made the commit a **seqlock**, a pattern where a generation number tells a reader whether the data changed underneath it:

```
s->generation++;           /* odd: write in flight */
s->config = *newconfig;
s->generation++;           /* even: committed */
```

A reader takes the generation, copies the configuration, and checks the generation again. An odd number, or a changed one, means a write overlapped the read. Chapter 9 records one reader edge case, found by reading the code.

A cleared screen, as the firmware gives

The same commit zero-fills a *fresh* allocation — a new base, virtual size or depth — because the real firmware hands out a cleared framebuffer. Without it, the window showed whatever was in guest RAM between the mode being set and the first paint: the boot began with a screen of garbage. A pure pan reuses the allocation and keeps its contents.

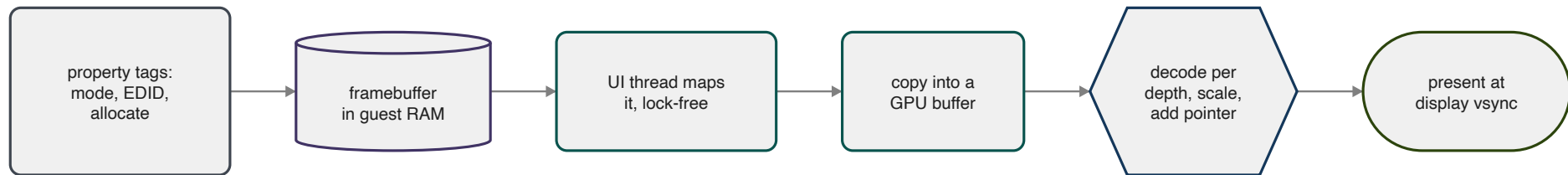
How the guest draws

Everything RISC OS draws ends up as bytes in that buffer. It gets there by four different routes, and they matter because later chapters move some of them onto the host:

What is drawn	Route on a stock system	With this fork
text and lines	the kernel writes pixels directly	unchanged
rectangle copies: drags, scrolls	the video driver, through the DMA controller in 2D mode	DMA model executes rows whole, on the host
rectangle fills: backgrounds	the video driver declines, so the kernel loops in ARM	the blitter device , via GVFill
sprites: icons, pictures	SpriteExtend compiles and runs ARM code	the blitter device , via GVFill

The mouse pointer is different again. It is never drawn into the buffer at all (chapter 4).

The display pipeline



The guest's own timing does not come from the window. Its vertical sync and half-frame pulses come from a generator on the fork's timer thread, 30 Hz by default. The front end's render loop sends the guest nothing, so the guest keeps the same time whether the window is visible, minimised or absent.

Input goes back the same way

The front ends turn host keyboard and mouse events into QEMU input events:

- keys as scan codes, for the emulated USB keyboard
- the pointer as **absolute** coordinates, for an emulated USB tablet

That needed one QEMU fix, covered in the [QEMU A72 walkthrough](#). RISC OS switches every HID device to report protocol, and QEMU's tablet had refused the request. Each input call takes QEMU's big lock only for the microseconds the input API needs.

2. The window owns the thread

The fork was first built on Windows. QEMU's own display back ends convert the guest's framebuffer on the CPU and hand it to GTK or SDL. The fork replaced that with a window it owns outright: `-display dx11`. This chapter walks through that front end — its threads, the boundary between C and C++, the swap chain that cannot wedge, the per-frame upload and decode, and the input path back to the guest.

Threads, and who may touch what

QEMU is a program, not a library, but it has a documented way for a display back end to take the main thread. It was added for the Cocoa port. `system/main.c` runs `qemu_init()` on the main thread; if the back end has set the `qemu_main` function pointer, QEMU moves its main loop to a new thread and calls that instead. The Windows front end uses exactly that hand-off:

Thread	Owns	Touches QEMU state
UI (the main thread)	the window, Direct3D 11, input state	reads guest RAM and the framebuffer config without locks; takes QEMU's big lock only to inject input
<code>qemu_main</code>	QEMU's main loop	everything, as usual
four vCPU threads	the emulated Cortex-A72s	everything, as usual

The design record states the rule for the frame loop plainly: the UI **never holds the big lock while rendering**. A frame reads a few words of configuration, reads the framebuffer bytes, uploads them and draws. The guest writes its screen under no lock either, and the design record accepts the consequence: a torn frame is exactly what a real monitor shows mid-update. Chapter 7 is about what happened when that acceptance was tested.

A C boundary for a C++ window

Direct3D is miserable from C and ordinary from C++. QEMU's headers do not parse as C++. So the front end is split in two at an `extern "C"` boundary:

File	Language	Holds
ui/dx11.c	C	QEMU glue: the framebuffer view, input injection, options
ui/dx11.cpp	C++17	the Win32 window, Direct3D device, shaders and frame loop
ui/dx11.h	both	the boundary: plain structs and functions, no QEMU types

The heart of the boundary is one struct. The glue fills it once per frame on the UI thread, and the C++ side draws from it:

```
typedef struct Dx11FbView {
    uint32_t xres, yres;           /* visible size, pixels */
    uint32_t xoffset, yoffset;     /* pan within the buffer */
    uint32_t pitch;               /* bytes per buffer row */
    uint32_t bpp;                 /* guest pixel format, bits */
    uint32_t pixo;                /* 1 = RGB order, 0 = BGR */
    uint32_t rows;                /* mapped buffer rows (>= yres+yoffset) */
    uint32_t generation;          /* config generation of this view */
    uint64_t gbase;               /* guest address of the buffer */
    const void *fb;               /* raw framebuffer bytes, pitch * rows */
    const void *palette;          /* 256 * 4 bytes, 0x00BBGGRR; always set */
} Dx11FbView;
```

The pointers are host addresses into guest RAM. There is no copy at this boundary.

Mapping the screen once per mode

The glue maps the framebuffer and palette with `address_space_map` and keeps the mapping until the configuration's generation moves:

```
gen = bcm2835_fb_get_config(view.fb, &cfg);
if (!cfg.xres || !cfg.yres) {
    return 0;                /* no mode programmed yet */
}
if (view.mapped && gen != view.generation) {
    address_space_unmap(&view.fb->dma_as, view.fb_ptr, view.fb_len, false, 0);
    address_space_unmap(&view.fb->dma_as, view.pal_ptr, 256 * 4, false, 0);
    view.mapped = false;
}
if (!view.mapped) {
    ...
    view.fb_ptr = address_space_map(&view.fb->dma_as, cfg.base, &fb_len,
                                   false, MEMTXATTRS_UNSPECIFIED);
    view.pal_ptr = address_space_map(&view.fb->dma_as,
                                   view.fb->vcram_base, &pal_len,
                                   false, MEMTXATTRS_UNSPECIFIED);
    ...
}
```

Two details matter. The mapping covers exactly what the guest indexes: the visible rows plus any pan offset. And pan offsets count only when the virtual size exceeds the visible one — the same rule the framebuffer model applies, mirrored here so the image does not shift.

A swap chain that cannot wedge

The first version presented with a plain blocking `Present(1)`, and it hung the whole UI in a way worth recording. A flip-model present blocks until the Desktop Window Manager (DWM) retires a buffer. If the window stops being composed for five seconds — occluded, minimised, or a DWM hiccup — Windows marks it *Not Responding*, DWM stops flipping it, and the one thread that pumps messages is stuck forever. A debugger found the thread parked inside DXGI.

Commit `a59d153d17` makes the chain **waitable**:

```
scd.SwapEffect      = DXGI_SWAP_EFFECT_FLIP_DISCARD;
```

```

scd.Flags          = DXGI_SWAP_CHAIN_FLAG_FRAME_LATENCY_WAITABLE_OBJECT;
...
HRESULT hr1 = swap2->SetMaximumFrameLatency(1);
dx11.swap_wait = swap2->GetFrameLatencyWaitableObject();

```

The frame loop then waits for a free latency slot *with the message queue armed*, and presents only when a slot is actually free:

```

if (dx11.swap_wait) {
    DWORD w9 = MsgWaitForMultipleObjectsEx(
        1, &dx11.swap_wait, 100, QS_ALLINPUT, MWMO_INPUTAVAILABLE);
    if (w9 != WAIT_OBJECT_0) {
        continue;          /* message or timeout: no free slot */
    }
} else {
    Sleep(1);
}
if (dx11_acquire_target()) {
    if (!dx11_render_frame()) {
        dx11.context->ClearRenderTargetView(dx11.rtv, clear);
    }
    HRESULT hr = dx11.swap->Present(1 /* vsync */, 0);
    ...
}

```

Input keeps flowing while the display is stalled. A hardware device is tried first, then WARP, Direct3D's software renderer. Drivers without the waitable interface fall back to the older blit-model chain: its present copies rather than flips, so it cannot wedge, and it paces on a sleep. A minimised window presents nothing.

Upload: two buffers, not one

Each frame copies the guest's bytes into a GPU buffer. The front end keeps **two** copies, and a comment in the source explains why:

A copy the guest wrote into while we were reading it splices two guest states together, and during a window move that puts part of the window where it used to be.

So the fresh copy goes into the buffer that is *not* on screen, and becomes the shown one only if it came out whole. Chapter 7 covers how "whole" is decided.

Each buffer is a Direct3D `ByteAddressBuffer` of the whole framebuffer, filled with one `Map(WRITE_DISCARD)` and one `memcpy`. It was a texture at first, until wide modes hit the texture ceiling. A `Texture2D` is limited to 16,384 texels across. A 3840-wide 32 bpp mode has a 15,360-byte pitch, just inside the limit, and anything wider overflowed. A flat byte buffer has no ceiling, and the pitch can be anything.

Decode: one shader source, seven decoders

The raw bytes are turned into colour on the GPU, at the guest's own resolution. One HLSL source is compiled at start-up into seven pixel shaders, for 32, 24, 16, 8, 4, 2 and 1 bits per pixel, selected by a `BPP` macro. The 32 and 16 bpp cases show the shape:

```
uint fb_word(uint row, uint byte_off)
{
    return raw.Load(row * P.dim.z + (byte_off & ~3u));
}

float4 ps_main(VSOut v) : SV_Target
{
    uint x = (uint)v.pos.x;
    uint y = (uint)v.pos.y;

    /* visible pixel -> buffer coordinates, pan included */
    uint bx = x + P.misc.x;
    uint by = y + P.misc.y;

    uint r = 0, g = 0, b = 0;
    #if BPP == 32
        uint w = fb_word(by, bx * 4);          /* aligned by construction */
```

```

    r = w & 0xff;
    g = (w >> 8) & 0xff;
    b = (w >> 16) & 0xff;
    if (P.misc.z == 0) { uint t = r; r = b; b = t; } /* BGR order */
#elif BPP == 16
    uint w = fb_word(by, bx * 2); /* two pixels per word */
    uint sh = (bx & 1u) * 16; /* odd pixel is the high half */
    w >>= sh;
    r = ((w >> 11) & 31) * 255 / 31;
    g = ((w >> 5) & 63) * 255 / 63;
    b = ((w >> 0) & 31) * 255 / 31;
#else
    /* palettised: 8 bpp, or sub-byte 1/2/4 with LSB-first packing */
    uint bpp = P.dim.w;
    uint byte = fb_byte(by, bx * bpp / 8);
    uint idx = (byte >> ((bx * bpp) & 7)) & ((1u << bpp) - 1);
    float4 c = pal.Load(int3(idx, 0, 0));
    return float4(c.rgb, 1);
#endif
    return float4(r / 255.0, g / 255.0, b / 255.0, 1);
}

```

A byte buffer loads whole words, so each depth extracts its bytes from aligned words. The palette is a 256×1 texture, refilled only when its bytes change. In practice the video driver only ever asks for 8 and 32 bpp. The other decoders come free, and all seven were checked pixel-exact on synthetic buffers.

Scale: the window is the monitor

A second pass stretches the decoded image over the whole client area. The design record puts it this way: the window is the monitor, so every mode fills it, as a Pi's GPU fills a real display. Three scalers are offered with `-display dx11,scaling=sharp|linear|nearest`, plus optional scanlines:

```

#if SCALING == 1
    /* Sharp bilinear: within each source texel the bilinear transition
     * is narrowed to a 1/ratio-wide band at the texel edge, so at 1:1

```

```

    * the image passes through untouched and at 2x+ it is crisp with
    * just enough filtering to avoid hard staircases. */
float2 ratio = max(outPx / srcPx, 1.0);
float2 halfw = 0.5 - 0.5 / ratio;          /* half the band width */
float2 i = floor(st);
float2 f = st - i - 0.5;                   /* -0.5..0.5 in-texel */
st = i + 0.5 + clamp(f, -halfw, halfw);
#elif SCALING == 2
    st = floor(st) + 0.5;                  /* nearest */
#endif

```

One Direct3D contract cost a black screen. The decoded surface has a full mip chain for downscaling, but it had been created without the `GENERATE_MIPS` flag. `GenerateMips` is silently a no-op without it. The moment the window was smaller than the guest mode, the sampler picked a mip level that had never been written, and the frame went black (commit `21db90950e`).

Input: scan codes and an absolute pointer

Keys. Windows delivers a scan code in each key message, and QEMU's AT set 1 keymap is indexed by exactly that. The first version indexed QEMU's *win32* keymap instead, which is keyed by virtual-key codes. Scan code `0x2c` is `z`, but virtual key `0x2c` is Print Screen. So typing `z` opened NetSurf's print dialogue, and later letters rewrote the URL bar. That was the "desktop freezes after a few keys" of the early bring-up (commit `c84d1bbc7d`).

The fix keys the lookup on the scan code and its `0xE0` prefix, and injects under the big lock:

```

uint32_t scancode = (lparam >> 16) & 0xff;
uint32_t code = (lparam & (1u << 24)) ? (0xe000 | scancode) : scancode;
...
unsigned int lnx = qemu_input_map_atset1_to_linux[code];
...
bql_lock();
qemu_input_event_send_key_linux(NULL, lnx, down);
bql_unlock();

```

The pointer is absolute. Ungrabbed, the host cursor's position in the client area is scaled onto the guest screen, so the two arrows stay together. Grabbed, host deltas accumulate into a position that is still sent absolutely. Motion feels relative, but it cannot drift, and releasing the grab realigns the arrows.

Three buttons, all for the guest. RISC OS is built around Select, Menu and Adjust, and Menu is the *middle* button, the one the desktop uses most. A front end that kept the middle button for itself would leave the guest unable to open a menu. So all three buttons go to the guest, and the grab moved to `Ctrl+Alt+G`. Losing focus always releases it.

What else the window does

Feature	How
full screen	Alt+Enter, borderless, never sent to the guest
DPI	per-monitor aware; the default window is 800×600 at the system scale
auto-resize	once, to the guest's settled mode, after about 3 s; then the size is the user's
screenshots	Print Screen writes the decoded surface to a PNG
snapshots	<i>Load snapshot</i> in the system menu, run as a bottom half on QEMU's loop
mode changes	a new generation rebuilds the per-mode pipeline without moving the window

The acceptance test compared the window's decode with QEMU's own screendump of the guest framebuffer. **84 of 480,000 pixels differed**, and all of them were the clock, which had ticked between the two captures.

3. The same program, another idiom

When the fork reached the Mac, the emulation needed no changes. What it needed was a window. Commit `7a39b30571` added `ui/metal.{h,c,m}`: the Direct3D 11 front end rebuilt in the Mac's own idiom, deliberately the same program. This chapter covers what carried over and what changed. It also covers the one thing the Mac front end grew that Windows has no twin for: an Apple Events surface designed for AI agents to drive the machine.

One program, two platforms

The Metal file's header states the design intent. Each frame, the UI thread pulls the framebuffer view over a narrow C boundary, uploads the raw bytes, decodes them with a per-format shader, and stretches the result over the whole view. Only the platform mechanisms differ:

Concern	Windows, ui/dx11.*	Mac, ui/metal.*
language	C++17 behind extern "C"	Objective-C, no ARC, behind a C header
shaders	HLSL, compiled at start-up with d3dcompiler	Metal Shading Language, compiled with newLibraryWithSource:
per-format variants	preprocessor macros	Metal function constants: BPP, SCALING, SCANLINES
raw framebuffer	two dynamic byte buffers	a ring of three shared-storage buffers

One program, two platforms, continued

Concern	Windows, ui/dx11.*	Mac, ui/metal.*
pacing	waitable swap chain	CAMetalLayer with display sync, plus an in-flight semaphore
thread hand-off	QEMU's qemu_main pointer	the same hand-off, which the Cocoa port introduced

The Metal file includes no QEMU headers at all. It talks to the emulation only through `ui/metal.h`, the twin of `ui/dx11.h`.

Memory the GPU already sees

On Apple Silicon, CPU and GPU share memory. The Metal front end uses that directly: the raw framebuffer bytes go into buffers created with shared storage, so the upload is **a memcpy into memory the GPU already sees**:

```
/* The raw guest bytes, a ring deep enough that the frame the GPU is
 * still reading is never the one the CPU is writing. Shared storage
 * means the "upload" is a memcpy into memory the GPU already sees --
 * on Apple silicon there is nothing further to transfer. */
for (i = 0; i < METAL_RING; i++) {
    fb.raw[i] = [m.device newBufferWithLength:bytes
                                options:MTLResourceStorageModeShared];

    ...
}
```

The ring is three deep (`METAL_RING`). Metal command buffers complete asynchronously, so the buffer the GPU is still reading must never be the one the CPU is writing. A semaphore limits frames in flight. The decoded surface is a private-storage texture at the guest's resolution: the scale pass's only input, and what a screenshot reads back.

On an Intel Mac with a discrete GPU, "shared" memory implies a transfer across the bus. The graphics design notes that its unified-memory reasoning may not hold there. **INFERRED:** the per-frame cost on such a machine has not been measured.

Mac input

A Mac mouse usually has one or two buttons, and RISC OS needs three. The front end maps them with modifiers:

Host action	RISC OS button
click	Select
Control-click	Menu
Command-, Option- or Shift-click	Adjust
a real middle button	Menu

The modifier is withheld from the guest during the click, so Control-click does not also send Control. The host cursor is hidden by a rule rather than by a grab: the guest draws its own arrow exactly under the host's, so the host arrow is hidden while the pointer is over the view in a key window. On Windows both arrows look alike and nobody notices; on the Mac they do not.

One Mac pitfall is recorded so it is not repeated. An early build skipped drawing while the window's `occlusionState` said it was hidden, and on the first run nothing was ever drawn.

An app, not a command line

A command-line QEMU is fine for a developer and awkward for everyone else. The Mac port wraps the binary in an application bundle:

- `tools/make-bundle.sh` copies the binary, an `Info.plist`, the generated scripting dictionary and the icon into `RISCOSQEMU.app`, then ad-hoc signs and verifies it. The icon is original geometry.
- `tools/run-app.sh` launches it through `open`, with CoreAudio and no QMP socket. The script calls that the *user persona*.

Apple Events, agents first

The Mac front end grew a control surface that Windows has no twin for. It is designed, in the fork's own words, **for an AI agent as the first and primary scripter**, with humans as a pleasant side effect. The design opens with a principle modelled on the emulator's:

rather than exposing QEMU, we should expose the functions needed to control and debug this machine ... be as curated and typed as possible, and expose raw QEMU only if we are desperate. We do need a consent model though — the whole surface is running off trust.

Why Apple Events rather than QMP

QMP already exposes everything. The design's case is about what an unattended agent lacks with a bare socket:

Need	QMP over TCP	Apple Events
a client	one that speaks QMP's greeting and capabilities exchange	osascript, already on every Mac, including JavaScript for Automation
identity	a port number tied to one process	the bundle identifier, which survives restarts
consent	none	macOS asks the <i>sender</i> once, keyed to both identities
discoverability	read the QEMU docs	a machine-readable dictionary (sdef) of every command

How it is built

The commands live in one C table in `ui/metal.c`. The generator `mksdef.py` turns that table into the scripting dictionary and a JSON description, so the interface and its documentation cannot drift apart. There are 24 commands in suite `MQem`. Each replies with a JSON envelope. Each is marked with one of two execution classes:

- **fast** runs synchronously under QEMU's big lock, on the UI thread, for reads and counters
- **bh** schedules a bottom half on QEMU's main loop and waits on a semaphore, for anything that changes the machine

Two rules keep the semaphore wait free of deadlock: the waiting UI thread holds no lock, and a bottom half never calls back into the UI thread. Commands are single-flight: a second caller gets `busy`.

Measured

Question	Result
Q1: are events delivered through the hand-run event pump?	yes, after one <i>Allow</i> click
Q2: latency	16.7 ms per warm Apple Event round trip, against 0.1 ms for QMP
Q3: does consent survive a rebuild?	yes, once measured, for an ad-hoc-signed bundle

The factor of about 160 settled the division of labour. Single commands for control and inspection go over Apple Events. Chatty loops, such as input streams and polling, stay on QMP.

Two kinds of screenshot

The surface separates two questions that a single screenshot command would blur:

Command	Returns	Time
screendump	the raw guest framebuffer — <i>the truth, no decode</i> — without the pointer	40 ms
screenshot	the decoded Metal surface, with the pointer sprite blended in	72 ms

A display bug shows up as a difference between the two.

Lessons from building it

- **One instance, or the wires cross.** A relaunch while an old instance still ran left two apps with one bundle identifier. Apple Events reached one process while QMP answered from the other, and the surface looked hung.

- **Never walk the object tree per frame.** A crash sample showed the pointer read resolving the VCHIQ device through QEMU's whole object tree on every frame: milliseconds per frame, and a lock-free read of a tree other threads own. The device pointer is now cached once.
- **The standard quit event was a land mine.** It fell through to the default application terminate, with no disc write-back. It now takes the same clean power-off path as closing the window.

Breakpoints and single-step over Apple Events, planned as stage E4, are not built.

The Intel Mac

The same code builds for Intel Macs, with no `arm64` conditionals. Chapter 7 of the [QEMU A72 walkthrough](#) covers the dependency build. The display acceptance test on a Xeon W-3235 compared a Metal surface capture with a QMP screendump taken a second apart. They differed in **134 of 480,000 pixels**, a 12 × 22 block at dead centre: the pointer, caught mid-move. Every other pixel was identical.

4. The pointer is a sprite again

On a real Pi 4 the mouse pointer is not drawn into the framebuffer. RISC OS's video driver asks the GPU to place a small image — a *display element* — on top of the screen, and the GPU composites it at scan-out. That is what makes a hardware pointer cheap: moving it touches no pixels.

While the VCHIQ peer refused every service, RISC OS fell back to a software pointer. The kernel painted the arrow into the framebuffer itself, saving the pixels underneath and restoring them around every plot while the pointer was on screen. Commit `9d23dc53c2` made the pointer a hardware pointer again, with the host as the hardware.

What the ROM sends

The ROM's hardware-pointer code talks to the GPU's *dispmanx* display manager over the VCHIQ service `'DISP'`. The fork's peer implements only the subset the pointer uses:

Command	What the ROM means	What the peer does
display open, get info	which display, and how big is it?	mints a handle; answers the display size
resource create	a 32 × 32 ARGB image	mints a resource handle
bulk write	here are the image's pixels	gathers them from guest pages into a staging copy
update start	begin a transaction	starts a fresh one

What the ROM sends, continued

Command	What the ROM means	What the peer does
element add, on the pointer layer	put the image here	records the destination rectangle, pending
element change attributes	move it	records the new rectangle, pending
element remove	hide it	records <i>not visible</i> , pending
update submit	commit the transaction	applies image and rectangle atomically

Every other command is refused, and only an element on the pointer layer is accepted. The image bytes arrive by the same page-list walk that chapter 8's sound path built.

Commit at update submit

The key property is atomicity. Nothing the ROM sends during a transaction is visible until it submits:

```
static void disp_commit(BCM2835VchiqState *s)
{
    if (s->disp_stage_len) {
        memcpy(s->ptr_image, s->disp_stage, s->disp_stage_len);
        s->disp_stage_len = 0;
    }
    if (s->disp_tx_pending) {
        s->ptr_visible = s->disp_tx_visible;
        s->ptr_x = s->disp_tx_x;
        s->ptr_y = s->disp_tx_y;
        s->ptr_w = s->disp_tx_w;
        s->ptr_h = s->disp_tx_h;
        /* the sprite's own resolution: the resource's, never the dest
         * rect's -- the plane is mode-independent and the rect may be
         * scaled by the ROM's display arithmetic */
    }
}
```

```
s->ptr_img_w = s->disp_res_w;
s->ptr_img_h = s->disp_res_h;
s->disp_tx_pending = false;
...
}
...                               /* then the generation is bumped once */
}
```

The compositor can therefore never see half a move: the new image with the old position, or the other way round.

Reading the pointer from the UI thread

Both front ends read the committed pointer every frame, from the UI thread, without QEMU's lock. The read uses the same seqlock discipline as the framebuffer configuration:

```
gen = qatomic_read(&s->ptr_gen);
smp_rmb();
out->generation = gen;
out->visible = s->ptr_visible;
out->x = s->ptr_x;
out->y = s->ptr_y;
...
out->argb = s->ptr_image;
smp_rmb();
out->stale = qatomic_read(&s->ptr_gen) != gen;
```

A stale read — one that raced a commit — tells the caller to keep the previous frame's pointer rather than draw a mixed one.

INFERRED: there is one small window this does not close. The staleness check happens before the caller copies the 64 × 64 image out of `ptr_image`. A commit that lands between the check and the copy could tear the sprite for a frame.

Three lessons from the wire

The Mac front end composited the pointer first. The graphics design records three things the wire taught, each of which cost a build:

- 1. The sprite is independent of the screen mode.** The first build indexed the image using the destination rectangle's size. But the ROM scales that rectangle with its own display arithmetic, so the indexing strode diagonally across the 32-pixel image, and the arrow came out as sheared garbage. The image is now indexed by the resource's own resolution and stretched to whatever rectangle arrives.
- 2. The rectangle lives in display space.** The ROM asks for the display's size before the first framebuffer exists. So the peer's answer is what the ROM's scale and offset arithmetic runs against, and rectangles arrive in *display* pixels. The compositor maps them back through the display space it answered. That is the honest real-hardware meaning: the firmware scales the desktop into the display, and the element sits in the display.
- 3. A triangle through the exact corner loses half the sprite.** The pointer quad is drawn with one covering triangle. A triangle whose long edge passes exactly through the rectangle's corner drops everything past that diagonal, under the rasteriser's fill rule. The busy timer kept its upper-left half, and the arrow lost most of itself. The covering triangle now overshoots the rectangle, and the overhang is discarded in the fragment shader.

The shape data was verified before any geometry was trusted. The pixels are little-endian `0xAARRGGBB` words. Those derive from the kernel's palette encoding through the ROM's byte-reverse-plus-alpha, and 134 texels of the 32 × 32 image are opaque arrow. A boot from the card sends 69 shape updates and about 356 move transactions.

Two display-space bugs

The answer that was asked too early

The display size the peer reports is what the ROM scales everything by. At first the peer answered with the *current framebuffer* size. The ROM asks once, during boot, while the machine is still in its 640 × 480 start-up mode, and then scales its pointer by that answer for the rest of the session. At 800 × 600 the error was a barely visible 1.25×. At 1920 × 1200 the pointer came out 10 × 10 instead of 32 × 32, confined to a third of the screen, and moving in three-pixel steps.

The peer now answers with **the EDID's preferred mode**, which does not change during a session (commit `7ef62fa51f`).

The letterbox the ROM draws

RISC OS places its screen on the display the way a GPU would: one scale factor for both axes, the image centred, the rest left as margin. At 1280×1024 on a 16:10 display that is a scale of 1.172, with 210 pixels of margin each side. The compositor stretched the framebuffer over the whole window, but placed the pointer as a fraction of the *display*. So the pointer was confined to the middle 78% of the screen, in a box whose edges the user could not reach.

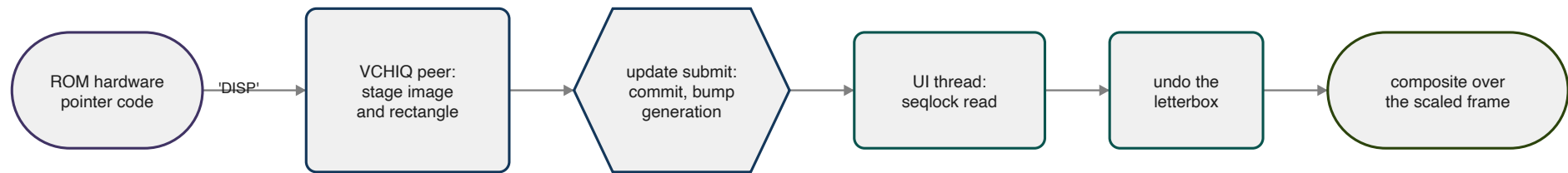
Commit [d2bb315d5b](#) inverts exactly what the ROM did. The Windows twin shows the transform:

```
static void dx11_disp_transform(double disp_w, double disp_h,
                                double fb_w, double fb_h,
                                double *scale, double *mx, double *my)
{
    double k = disp_w / fb_w < disp_h / fb_h ? disp_w / fb_w : disp_h / fb_h;

    if (!(k > 0)) {
        k = 1.0;
    }
    *scale = k;
    *mx = (disp_w - fb_w * k) / 2.0;
    *my = (disp_h - fb_h * k) / 2.0;
}
```

When the aspect ratios agree, the transform is the identity. That is why 1280×800 on a 1920×1200 display always worked, and that observation is what found the bug. It was verified by driving the tablet to all four corners over QMP: the rectangle RISC OS sent, spanning 186 to 1685 across, mapped back to guest pixels 0 to 1279 on a 1280-wide screen.

The pointer, end to end



The pointer is composited on top of the scaled frame, with straight alpha, so the ROM's anti-fringe fill behaves as it does against the real firmware. It stays sharp at any window scale. It is never in guest RAM: `screendump` shows the desktop without it, and `screenshot` blends it in.

5. Moving drawing to the host

Under emulation the slowest part of the machine is the CPU, and RISC OS does most of its drawing on the CPU. A large window repainting in visible pieces is the symptom. This chapter covers the host side of the fix: why it is not the host GPU, the device the guest talks to, and how that device performs fills, copies and sprite plots.

What RISC OS offers a video driver

RISC OS 5 gives its video driver a way to take over drawing: *GraphicsV reason 13, Render*. The kernel tries it before plotting in software. There are exactly three operations:

Operation	Issued for	The Pi's driver
NOP	before software plotting, to let a pending operation finish	waits for its DMA channel to go idle
CopyRectangle	window drags, scrolls, block copies, text scrolling	does it , with the DMA controller in 2D mode
FillRectangle	window backgrounds and borders, clear screen	declines ; the kernel fills each row in ARM

Sprites take a different path. The kernel hands plots to the *SpriteV* vector, where *SpriteExtend* generates a bespoke ARM routine for each sprite format and caches only eight of them. Under emulation all of that is guest CPU work.

The plan, and what shipped

The graphics design, written early on 11 September, planned a module called *FastSpr* for sprite plots. It would ring the HostFS doorbell device, and fill would be deferred. Its argument was that claiming FillRectangle means layering a filter driver over the ROM's video stack, work better done properly in the project's own ROM. It also planned an asynchronous Metal path. What shipped twelve hours later differs on three counts:

Planned	Shipped
sprites through the HostFS doorbell	a separate, register-programmed device , riscos-blitter
fill deferred to an own-ROM era	fill claimed after all , by an ordinary vector claim that checks the reason and operation
an asynchronous path on the host GPU	no GPU at all

Why not the GPU

On Apple Silicon the GPU shares memory with the CPU. It could fill the guest's framebuffer in place, with no transfer. So the obvious objection had to be *measured*, and `tools/fillbench.m` did it on an M4:

Rectangle	memset, per row	memset, one call	Metal, synchronous	Metal, pipelined
full screen, 9 MB	118 μ s	68 μ s	510 μ s	132 μ s
a window, 1.9 MB	49 μ s	—	232 μ s	27 μ s
an icon row, 6 KB	0.1 μ s	—	149 μ s	20 μ s

The benchmark's conclusion: **the cost is latency, not bandwidth**. A GPU dispatch costs about 150 μ s to submit and wait for. That cannot be pipelined away, because the blit has to be synchronous: the guest plots text and sprites into the same framebuffer on the very next instruction. Against a real redraw — thirteen small rectangles and two full-screen clears — that is about 81 μ s of `memset` against about 2.6 ms of Metal.

The benchmark still left a mark on the device. It showed that back-to-back rows want one big write rather than one per row, and `blit_contiguous()` came from that.

The device: riscos-blitter

The device is new, invented for this purpose, and models no real BCM2711 hardware. It sits at `0xFD404000`, one page past the HostFS doorbell, in the low peripheral window. The guest maps its page with `OS_Memory 13` instead of assuming an address. RISC OS builds its logical memory map from what the HAL asks for, so an unclaimed page simply aborts.

The registers

Offset	Register	Meaning
0x00	MAGIC	reads 'BLIT'
0x08	FEATURES	fill, copy, sprite, mask, table
0x0c	GO	write: run; read: the last status
0x10–0x14	OP, FLAGS	NOP, FILL, COPY or SPRITE; addressing flags
0x18–0x1c	DEST, SRC	framebuffer byte offset; source address
0x20–0x2c	WIDTH, HEIGHT, DSTRIDE, SSTRIDE	geometry; signed row strides

The registers, continued

Offset	Register	Meaning
0x30–0x40	PATTERN (4 words), PATLEN	a fill pattern, repeating every 1 to 16 bytes
0x44–0x58	DSTX, DSTY, CLIPX0...CLIPY1	sprite position and clip rectangle, top-left origin
0x5c–0x6c	BPP, MASK, MSTRIDE, SRCBPP, TABLE	sprite depth, mask and colour table

The status codes are `OK`, `BADOP`, `BADGEOM`, `NOFB` (no framebuffer), `RANGE` (a row fell outside the framebuffer) and `FAULT` (a source row would not translate).

Three design decisions, from the header

Registers, not a descriptor block. A block in guest RAM would be one MMIO write instead of eleven. But it would have to be named by *physical* address, and a RISC OS module holds a *logical* one. The module area is not identity mapped, so the translation costs an `OS_Memory` call per rectangle, with cache maintenance. The header's verdict: eleven writes cost about a microsecond, which is nothing beside the fill they replace.

Offsets, not addresses. With the `BLIT_F_FB` flag, the guest gives byte offsets *into the framebuffer*, and the host adds the base it already knows. That is the only addressing RISC OS needs here, and it saves the guest a translation. It also lets every row be checked against the framebuffer's extent, so a wrong rectangle **spoils the screen rather than memory belonging to something else**.

Sprite sources are guest-virtual. A sprite lives in a sprite area. RISC OS dynamic areas are virtually contiguous but physically scattered, so there is no one physical address to hand over. With `BLIT_F_SRC_VIRT`, the host walks the guest's page tables itself. That costs the guest nothing and handles the scatter for free.

Synchronous, with the guest stopped

A blit runs inside the MMIO write to `G0`, on the vCPU thread, with the guest stopped. The device therefore refuses geometry no screen could need, because a wild height would hang the guest rather than fault it:

```
#define BLIT_MAX_WIDTH      (1u << 16)
#define BLIT_MAX_HEIGHT    (1u << 14)
#define BLIT_MAX_BYTES     (64u << 20)
```

Fill

The fill checks the rectangle against the framebuffer, then takes the fastest of three routes. The common case is a pattern of one repeated byte: the white and grey window backgrounds. For that case a `memset` beats building a row:

```
solid = true;
for (uint32_t i = 1; i < s->patlen; i++) {
    if (pat[i] != pat[0]) {
        solid = false;
        break;
    }
}

blit_span(s, s->dstride, &lo, &span);
host = blit_map(dest + lo, span, &mapped);
if (host) {
    uint8_t *rowpat = NULL;          /* built on first unsolid row */

    for (uint32_t y = 0; y < s->height; y++) {
        uint8_t *to = host + ((int64_t)y * s->dstride - lo);

        if (solid) {
            memset(to, pat[0], s->width);
            continue;
        }
        ...
        memcpy(to, rowpat, s->width);
    }
}
```

```

    blit_unmap(host, mapped);
} else if (blit_contiguous(s, s->dstride)) {
    ...                               /* 1 MB chunks through the address space */
} else {
    ...                               /* a row at a time */
}

```

In order of preference:

1. **Map the whole strided span once** with `address_space_map`, and write rows with plain stores.
2. **If it will not map and the rows run back to back**, write in 1 MB chunks. That applies only when the pattern length divides the width, so a continuous run and a row-by-row fill put the same byte in the same place.
3. **Otherwise**, write one row at a time through QEMU's DMA write.

Unmapping a span also marks the gaps between rows as dirty, about twice the bytes actually written. A row-by-row variant that dirtied less was tried. A narrow fill then cost 1.00 μ s instead of 0.52 μ s, and neither front end reads the dirty bitmap, so it was reverted.

The range check needs only the first and last rows. The stride is constant, so rows run monotonically through memory. An earlier version walked every row: a thousand iterations to validate a full-screen fill.

Copy

A copy maps both spans. **Disjoint ends**, such as a copy between two screen banks, copy directly: one `memcpy` when contiguous. **Overlapping ends** — a window moved within the same framebuffer — bounce each row through scratch memory, or use one `memmove` when contiguous. The DMA paths remain as the fallback.

INFERRED, from the guest code: GVFill never issues a COPY. Window copies still go through the video driver's DMA path (chapter 6 of the [QEMU A72 walkthrough](#)). So the host copy is ready for the screen-banks plan in chapter 7, but no shipped guest code reaches it yet.

Sprite

A sprite plot is rows of pixels from guest RAM into the framebuffer. The host clips the sprite against the caller's clip rectangle and the screen, all inclusive, with a top-left origin. The header explains the choice: a rectangle intersection is easy to get right in C and fiddly in relocation-free ARM.

The expensive part turned out to be reading the source. Translating a row at a time, as a debug interface invites, is far too slow: a page-table walk per row. Mapping a page at a time was still slow: a megabyte of sprite is 256 maps. So the source reader **maps as much as the address space will give in one call**, then walks forward, extending the run while the guest's pages stay physically adjacent:

```
if (w->valid && va >= w->va_base + w->run_len) {
    /* Just past the run: try to grow it rather than remap. */
    uint64_t next = w->va_base + w->run_len;
    hwaddr pa;
    MemTxAttrs attrs;

    while (w->run_len + 0x1000 <= w->maplen &&
           va >= w->va_base + w->run_len &&
           blit_src_xlate(w, next, &pa, &attrs) &&
           pa == w->pa_base + w->run_len) {
        w->run_len += 0x1000;
        next += 0x1000;
    }
}
```

`blit_src_xlate` is a wrapper around QEMU's `cpu_translate_for_debug`, which walks the guest's page tables without faulting. As the comment beside the walker puts it, translation is the cheap half and mapping is the half worth avoiding.

The destination span is mapped only when the rows nearly touch, when $w \times \text{bpp} \times 2 \geq \text{pitch}$. Otherwise rows are written one at a time, which dirties exactly what changed. Paths for masked sprites and for packed sources through a colour table exist on the host, but chapter 6 explains why the guest does not use them.

GO, and the damage word

Everything funnels through one function, run by the write to GO:

```
static void blit_go(RISCOSBlitterState *s)
{
    int64_t t0 = g_get_monotonic_time();
    uint32_t rc;

    if (!blit_geom_ok(s)) {
        rc = BLIT_RC_BADGEOM;
    } else if (s->op == BLIT_OP_SPRITE) {
        rc = blit_sprite(s);          /* geometry is in pixels, not bytes */
    } else if (s->width == 0 || s->height == 0) {
        rc = BLIT_RC_OK;              /* nothing to do, but not an error */
    } else {
        ...                          /* FILL, COPY, SPRITE or BADOP */
    }

    if (rc == BLIT_RC_OK && s->width && s->height) {
        riscos_blitter_note_damage();
    }
    trace_riscos_blitter_go(s->op, s->width, s->height,
                           s->dstride, s->sstride, rc,
                           (uint32_t)(g_get_monotonic_time() - t0));

    s->status = rc;
}
```

The trace point times every blit, and produced most of the numbers below. And `riscos_blitter_note_damage()` sets one atomic word, which the front ends take and clear with an atomic exchange: chapter 7 is about what they do with it.

Measured, step by step

The optimisation sequence is recorded commit by commit. It is a good example of measuring before each change:

Step	Sprite plot, guest vs host	Commit
first host plot	SpriteExtend 460 μ s, host 110 μ s: 4.2x	bbb1aefe06
framebuffer device found once, not per call	about 40 μ s per call gone	fce3b29fb1
coalesced source maps; VDU constants cached	490 μ s against 80 μ s: 6.1x	a3f6b6fafe
inside the 80 μ s	61.8 μ s host blit, about 18 μ s guest overhead	a3f6b6fafe
after the device review, on a Mac	about 61 μ s against about 490 μ s; <i>wants a rerun</i>	11ad1ee7c2

INFERRED: *SprBench times a thousand plots with the guest's centisecond clock, so it resolves 10 μ s per plot. A figure such as "about 61 μ s" is finer than that clock can measure, so treat the Mac figure as approximate until the planned rerun.

What the host takes on

The share below is 94.3%, although both READMEs still print 98.2%. That figure came from the masked-sprite build, which was backed out because it drew the desktop wrong (chapter 6).

Measure	Result	Commit
share of sprite pixels plotted by the host	94.3%	a3f6b6fafe, 8b1d43fb4b
correctness against the same scene without the module	0 of 2,304,000 pixels differ	a3f6b6fafe
one desktop redraw	15 fills, 2 of them full-screen	35eb1d8836
a session's fills	5,878, mean 108 KB, only 11 whole-row	c2b826a5f6
8 bpp window furniture	a 91 × 17 fill in 2 μs; a 1920 × 1200 erase in 303 μs	0223e6f2d8

6. GVFill, line by line

The blitter device does nothing on its own. Something inside RISC OS has to notice a fill or a sprite plot, decide whether the host can do it, program the device, and tell the kernel the job is done. That is **GVFill**, a RISC OS relocatable module of under a thousand lines of ARM assembly. This chapter walks through it, at version 1.03 on the published branch. It also tells two stories from its history: a feature that drew the desktop wrong and was backed out, and one register that nearly sank the ROM version.

Built with no relocations

GVFill is pure assembly. It is assembled with clang and turned into a module image with `llvm-objcopy`, and no RISC OS linker is involved. That imposes one hard rule, and the build script enforces it:

```
n=$(("$READOBJ" -r blitmod.o | grep -c R_ARM || true)
```

A module is loaded wherever the RMA (RISC OS's module area) has room, and nothing relocates it. So any relocation left in the object is a branch into nowhere. It does not fail at load time. As the README puts it, it takes the desktop black minutes later. So `build.sh` refuses to produce a module with any `R_ARM` relocation left.

Two consequences run through the source:

- **ADR reaches only about a kilobyte**, and the handler outgrew that twice. The second time it broke code that had previously assembled.
- So **data is reached through r12**, the workspace pointer that vector dispatch provides for exactly this purpose, as `[r12, #offset]` loads.

The module header

A RISC OS module begins with a table of offsets to its entry points. GVFill's is thirteen words:

```
_start:  
base:
```

```

.word    0                @ start
.word    init    - base
.word    final   - base
.word    sv_service - base    @ service call handler
.word    title   - base
.word    help    - base
.word    cmdtab  - base
.word    0                @ SWI chunk
...
.word    modflags - base
...
modflags:
.word    1                @ 32-bit compatible

```

Every entry is written as a label minus `base`, which is what makes the table position-independent.

Three star commands

The module adds three commands, for testing and measuring:

Command	What it does
*BlitFill	paints a 200 × 100 test rectangle through the device, without involving GraphicsV
*SprStats	sprite plots seen, taken and passed, by count and by area, and which gate declined the rest
*SprBench	1,000 plots of a 512 × 512 sprite on the host, then 1,000 through SpriteExtend

Workspace: nothing in the image is ever written

Version 1.03 keeps all writable state in a **172-byte structure claimed from the RMA at initialisation**. That is the shape the DDE gives C modules, written here in assembly. The module image itself is never written after the build, which is what makes it safe to place in read-only ROM. The private word carries the structure's address to every entry point.

The initialisation that claims it contains the trap that nearly sank this version:

```
@ The size goes in R3 and the block comes back in R2 -- ModHand's
@ RMAClaim_Chunk rounds R3 up for the heap, and a size left in the
@ wrong register claims a garbage-sized block whose neighbours our
@ zero loop then flattens. Found the hard way: the flatten took out
@ our own literal pool.
MOV    r0, #ModClaim
MOV    r1, #0
LDR    r3, =WS_END
SWI    XOS_Module
```

OS_Module 6 takes the claim size in R3, not R2. With the size left in R2, the module claimed a garbage-sized block. Its zeroing loop then flattened the neighbouring memory, including the module's own literal pool with the device's address in it. The device magic then read back as anything but 'BLIT', and initialisation reported no blitter. It was found by parking the mapped address and the value read into the structure, and reading them out of guest RAM (commit c0a55e4f44).

Initialisation

With the workspace claimed and zeroed, `init` maps the device and claims two vectors:

```
MOV    r0, #MapIOPermanent    @ OS_Memory 13
LDR    r1, =BLIT_PHYS         @ 0xFD404000
MOV    r2, #0x1000
SWI    XOS_Memory
MOVVS  r9, r0
BVS    init_undo
STR    r3, [r11, #WS_BLITLOG] @ the logical address
LDR    r2, [r3]
LDR    r4, =BLIT_MAGIC
TEQ    r2, r4
ADRNE  r9, err_nODEV          @ "No host blitter at this address"
```

```

BNE    init_undo

MOV     r0, #GraphicsV
ADR     r1, gv_handler
MOV     r2, r10           @ the private word
SWI     XOS_Claim
...
MOV     r0, #SpriteV
ADR     r1, sv_veneer
MOV     r2, r10
SWI     XOS_Claim

```

Claiming each vector with the private word as `r2` means every handler arrives with `r12` pointing at it. A failed initialisation undoes itself: it releases whatever it claimed and frees the workspace. `final` releases both vectors with the same values and frees the structure.

Running on a machine without the device fails cleanly. The magic check produces a proper RISC OS error instead of an abort.

The fill handler

GraphicsV carries every video driver call, so the handler's first job is to get out of the way fast. Only reason 13, Render, with operation 2, FillRectangle, is examined:

```

gv_handler:
    STMFD  sp!, {r0-r3, r5-r11, lr}    @ not r4: the answer goes there
    LDR     r12, [r12]                 @ workspace, via the private word
    AND     r5, r4, #0xFF
    TEQ     r5, #GraphicsV_Render
    BNE     gv_pass
    TEQ     r1, #GVRender_FillRectangle
    BNE     gv_pass

```

The colour block, found by looking

No video driver had ever implemented `FillRectangle`, so nothing settled what the colour parameter points at. The kernel's workspace header describes it one way; one of the two call sites builds something else. The fork settled it by observing both call sites on a live desktop (commit `35eb1d8836`). Both pass **sixteen words of interleaved (ora, eor) pairs**, and a fill computes `(dest ORR ora) EOR eor` for each word. The desktop's grey appeared as `FFFFFFFF/FF888888` pairs.

When every `ora` is all ones, that expression reduces to a constant. That is the only case the handler takes:

```

LDR    r6, [r2, #16]      @ the colour block
LDR    r9, [r6, #4]
MVN    r9, r9             @ r9 = the pixel word
MOV    r8, #8
gv_chk:
LDR    r0, [r6], #4
CMN    r0, #1             @ EQ only if 0xFFFFFFFF
BNE    gv_pass
LDR    r0, [r6], #4
MVN    r0, r0
TEQ    r0, r9
BNE    gv_pass
SUBS   r8, r8, #1
BNE    gv_chk

```

Any other GCOL action needs the destination read back, so it is left to the kernel.

Geometry, then eleven writes

The handler reads the screen's line length, depth and height with `OS_ReadVduVariables`, and declines depths below 8 bpp, where a pixel is not a whole number of bytes. It converts the inclusive, bottom-left-origin rectangle into a top-left byte offset and a size in bytes. Then it programs the device:

```

MOV    r1, #0P_FILL

```

```

STR    r1, [r0, #BLIT_OP]
MOV    r1, #F_FB
STR    r1, [r0, #BLIT_FLAGS]
STR    r11, [r0, #BLIT_DEST]    @ byte offset of the top left
STR    r3, [r0, #BLIT_WIDTH]    @ bytes
STR    r6, [r0, #BLIT_HEIGHT]    @ rows
STR    r5, [r0, #BLIT_DSTRIDE] @ one line length
MOV    r1, #4
TEQ    r9, r9, ROR #8
MOVEQ  r1, #1                    @ every byte equal: 8bpp plain colour
BEQ    gv_patlen
TEQ    r9, r9, ROR #16
MOVEQ  r1, #2                    @ halfwords equal: 16bpp plain colour
gv_patlen:
STR    r1, [r0, #BLIT_PATLEN]
STR    r9, [r0, #BLIT_PATTERN]
...
STR    r9, [r0, #BLIT_GO]

LDR    r1, [r0, #BLIT_GO]    @ status
TEQ    r1, #0
BNE    gv_pass                @ refused: let the kernel do it
...
MOV    r4, #GraphicsV_Complete
gv_pass:
LDMFD  sp!, {r0-r3, r5-r11, pc}

```

The pattern length is a neat detail. The device refuses a width the pattern length does not divide. So the handler reports the *shortest* length the colour word repeats at: one byte for an 8 bpp plain colour, which also lets the device use a `memset`. Odd widths stay fillable.

The status read is the whole contract. On success the handler returns `r4 = GraphicsV_Complete`, and the kernel skips its own plot. On any refusal it leaves `r4` alone, and the kernel does the fill as before.

The sprite handler

SpriteV carries every sprite operation, and **claiming the vector at all costs something**. The kernel takes an internal fast path only while it is the sole owner of SpriteV. Once GVFill claims the vector, every sprite operation goes down it, so the handler must be cheap to pass through and must leave every register alone. It takes one reason, 52, a scaled sprite plot, and only when every gate passes:

Gate	Why
output is the screen, not a sprite	a plot into a sprite belongs in that sprite
the sprite is given by pointer	not looked up by name
unmasked	masked plots are declined; see below
plot action is a plain store	other actions need the destination
no left-hand wastage	the first pixel starts on a word boundary

The gates, continued

Gate	Why
equal scale factors	not really scaling
sprite format matches the screen	32 bpp: type 6; 8 bpp: type 4; 16 bpp: type 5, only on a 5:5:5 screen
at 8 bpp, no private palette	byte indices only copy when both sides mean the same colours

The 16 bpp gate is subtle. The Pi's 16 bpp screen is 5:6:5, where old type-5 sprites are 5:5:5 and genuinely need conversion: a copy would re-tint them. So those go to SpriteExtend.

Two kinds of VDU variable

Reading VDU variables on every plot was a measurable share of the guest's overhead. The handler splits them. The comment in the source explains the split:

```
@ Five of these are mode constants and six are not: the graphics
@ window and origin are set per redraw rectangle, so the Wimp
@ changes them between one plot and the next. Read the constants
@ only when the mode or the output has changed under us.
```

The service handler marks the cached constants stale on `Service_ModeChange`. Since version 1.03 it also does so on `Service_SwitchingOutputToSprite`. A boot's first plots go *into a sprite*, and while output is a sprite, `OS_ReadVduVariables` answers with that sprite's geometry. Caching those values once left every later plot failing the depth gate. The same service call now also sets the *output is a sprite* flag, so such plots are passed.

Programming the device, and a claim by stack

After converting OS units to pixels, and the bottom-left origin to a top-left one, the handler programs a sprite blit. It hands over the sprite's **logical** address, and the host walks the page tables:

```
MOV    r11, #OP_SPRITE
STR     r11, [r10, #BLIT_OP]
MOV     r11, #F_SRC_VIRT      @ rows run top down, as the screen does
STR     r11, [r10, #BLIT_FLAGS]
LDR     r11, [r2, #spImage]
ADD     r11, r11, r2          @ logical: the host walks the page tables
STR     r11, [r10, #BLIT_SRC]
...
STR     r11, [r10, #BLIT_GO]
LDR     r11, [r10, #BLIT_GO]
TEQ     r11, #0
BNE     sv_pass               @ refused: leave it to SpriteExtend
```

Taking a plot off a vector needs a trick, because a vector handler normally passes on by returning. RISC OS's vector dispatch pushes the caller's return address before walking the chain. So *passing on* is an ordinary return, and *intercepting* is unstacking the handler's own frame and then returning straight to the caller:

```
@ Claim.  CallVector pushed the caller's return address before
@ walking the chain, so passing on is MOV pc, lr and intercepting is
@ taking that address off the stack once our own frame is gone.
LDMFD    sp!, {r0-r11, lr}
MSR      CPSR_f, #0          @ V clear: no error
LDMFD    sp!, {pc}
```

Every declined plot is also counted by *area*, not only by call. The split that matters is pixels, and pixels decide whether the next case is worth writing.

The feature that drew the desktop wrong

Masked sprites — icons with transparent edges — looked like the next four points of coverage. Commit [10ba10b52c](#) added them. A one-bit-per-pixel mask was merged on the host, whole groups of 32 pixels were moved at a time, and a pixel comparison came back clean. Coverage rose from 94.3% to **98.2%** of sprite pixels.

The bisection

Loading the module then corrupted the screen: parts of windows drawn in the wrong place, and content showing through where it should have been covered. A bisection against a reference redraw found where it started (commit `8b1d43fb4b`):

Module at	Pixels differing
a3f6b6fafe, before masks	0
10ba10b52c, masked sprites	484,513
1011ad8aea, packed sources added	421,534 (<i>contaminated</i>)
05cc1f5a8c, mask gated on the plot action	749,019 (<i>contaminated</i>)

The last two counts are contaminated: the Wimp does not repaint areas it believes are still valid, so damage carries over from one run to the next. The commit records that gating the mask on the plot action reduced the damage without curing it. **The module went back to a3f6b6fafe**, which measures identical to no module at all. The commit's verdict: masked sprites were worth about four points of coverage, and not worth a wrong screen.

Two theories died on the way, and were recorded so nobody retries them: the framebuffer's pan offset, and RISC OS drawing into a second screen bank. It allocates one, but the offset stays at 0,0 throughout, even while scrolling.

Plot-action bit 4

With masks gone, the declined plots were counted gate by gate: named 0, wastage 0, scaling 0, mask 0, depth 3, no table 0 (commit `05cc1f5a8c`). That leaves the plot action declining **56 of 59** calls. Their raw values are 16 and 24, so **bit 4 is set on every one**. SpriteExtend's plot compiler reads only the low four bits, so ignoring bit 4 looked safe. It is not: accepting those calls changed 3,468 pixels in a cluster of filer icons. Whatever bit 4 means is consumed somewhere between the SWI and that compiler, and until it is known, those plots are declined.

Soft-loaded, or in the ROM

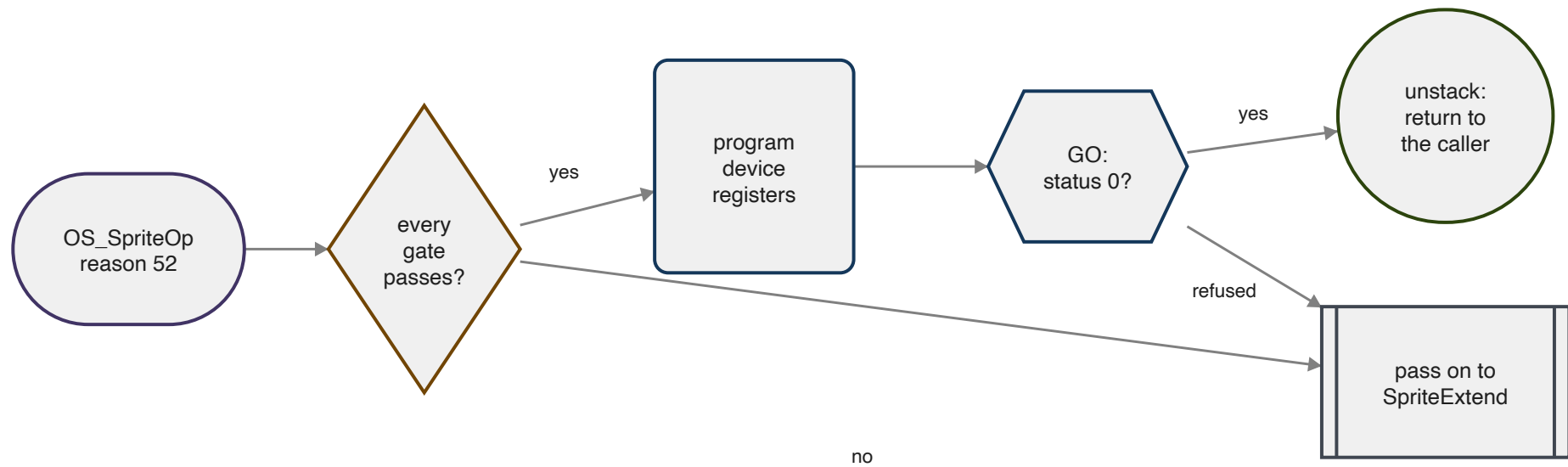
Version 1.03 runs either way, but the two are **not equivalent**:

Loaded	Fills	Sprites
soft-loaded from the host share at boot	yes	yes
spliced into the ROM with <code>tools/mkrom.py</code>	yes: 26 fills in one boot's trace, 1–2 μ s each	never sees a plot

In the ROM, something later in the boot claims SpriteV in front of GVFill and answers the plots itself. A vector's handlers are called most-recent-first, and the ROM is the earliest place to initialise. Killing eight likely claimants in turn changed nothing, and `*RMReInit GVFill` put it back in front. So the decision recorded on 13 September is that only HostFS and its filer go into the ROM. GVFill loads from the share's `$.Modules` directory before the desktop starts. The [HostFS walkthrough](#) covers that boot path.

`*RMKill GVFill` takes the module out of the drawing path instantly. The README names it as the first thing to try if anything on screen looks wrong.

A plot's path through GVFill



7. Knowing what the guest drew

A front end that copies the guest's screen on its own clock will sometimes copy it in the middle of a redraw. The result is a *torn* frame: part of a window at its old position and part at its new one, visible as a flash during drags. Four approaches were tried to decide when a copy is worth taking. This chapter follows them in order, ends with the design for the fix that would remove tearing altogether, and marks where that design is still only a plan.

Attempt 1: copy every frame, and accept the tear

The first Windows front end copied the whole framebuffer on every presented frame, unsynchronised. Its design record gives the reason: the guest writes its screen under no lock, and **a torn frame is exactly what a real monitor shows mid-update**. Dirty tracking was deferred until it was shown to matter.

The cost was measured before anything else was done. With the window's full pipeline live at 60 Hz, a guest benchmark ran about 2.3% slower, within the noise. The copy runs on the UI thread and the GPU, off the vCPU's core, so it does not cost the guest. What remained was purely a question of what the user saw.

Attempt 2: copy in step with the guest's frame

The next idea, commit `6a48d5b5a4`, came from the video driver's own behaviour. It holds pending screen updates and flushes them at the half-frame pulse. So the vsync generator was taught to report its phase, and the front end copied only in the first half of each guest frame, after one flush and before the next. Uploads fell to exactly 150 per 300 presented frames: a 60 Hz window against a 30 Hz guest.

It was backed out twenty minutes later, in commit `c5f7bb86b7`, and the reason is worth keeping:

- **The premise was wrong.** The Wimp does not paint on vsync. RISC OS multitasks cooperatively, so a task repaints when it next polls, driven by the centisecond tick and by mouse events. Only the pointer, an explicit vsync wait and screen-bank flips are tied to vsync. There is no quiet phase in the frame to aim a read at.
- **The effect was worse than nothing.** Sampling once per guest frame held each captured frame on screen for two host frames, so every torn capture stayed up twice as long.

The phase function was kept, because it correctly describes the generator and costs nothing when unused. A future screen-bank flipper will want it.

Attempt 3: fingerprint the screen

The Windows front end then went back to reading every frame, and added a probe. It fingerprinted a spread of rows either side of each copy. If the fingerprints differed, the guest had written during the copy. That copy went into the buffer not on screen and was discarded, and the previous whole frame stayed up. At rest, about **5 copies in 3,000** were splices.

It worked, but it guessed at something another component already knew exactly.

Attempt 4: ask the device that drew it

The blitter device already knows when the guest draws, because much of the drawing now goes through it. Chapter 5 showed the one line that records it: `riscos_blitter_note_damage()`, an atomic word set after every blit that lands. The Metal front end adopted it first, in commit `6c26b418f0`, and Windows followed in commit `b3d60dbe6a`, which also deleted the probe.

Why not QEMU's dirty bitmap

QEMU already tracks which pages of guest RAM change, and that would be complete. But reading it needs QEMU's big lock, and the front end runs on its own thread. Three attempts at it triggered assertions in three different places inside QEMU's locking and memory-transaction code. One of those came from a view function being reached both with and without the lock held. An atomic word needs no lock at all.

The measurement that justified it

Over five thousand frames, **0.1% of rows changed between one frame and the next**. The front end was moving nine megabytes thirty times a second to show the same picture. The desktop is idle almost all the time, because the Wimp draws only when a task asks it to.

The settle rule

Both front ends apply the same rule: **copy when the guest has drawn and then stopped**. Seeing the damage flag means the guest is mid-redraw, so the previous whole frame stays on screen. Seeing it stop means the screen has settled, and that is the moment to copy. This is the Metal version:

```
#define METAL_SETTLE_MAX 4          /* frames to wait for a settle */

static bool fb_upload(const MetalFbView *v)
{
    static unsigned held;
    static bool pending;
    bool drawing = riscos_blitter_take_damage() != 0;
    bool settled;

    if (drawing) {
        pending = true;          /* new content, not shown yet */
    }

    settled = pending && !drawing;
    if (fb.uploaded && !settled && ++held < METAL_SETTLE_MAX) {
        ...                    /* a periodic log line */
        return false;
    }
    ...                        /* count settles and timeouts */
    held = 0;
    pending = false;

    fb.ring = (fb.ring + 1) % METAL_RING;
    memcpy([fb.raw[fb.ring] contents], v->fb, (size_t)v->pitch * v->rows);
    fb.uploaded = true;
    ...
    return true;
}
```

`METAL_SETTLE_MAX` bounds the wait at four frames, for two reasons given in the source. Continuous drawing, such as a drag or a scroll, never settles and still has to animate. And text and lines are plotted straight into memory without passing through the blitter, so nothing raises the flag for them. Without a forced copy, typing would never appear.

The Windows twin, with three differences

The Windows version applies the same rule, with three deliberate additions:

1. **It gates only once the blitter has been seen.** A guest running without GVFill never raises the flag. Gating on it anyway would drop the whole display to the four-frame timeout for a guest drawing perfectly normally. The commit notes the same hazard is present on the Mac and worth fixing there.
2. **It re-checks after the copy.** If the flag rose while the bytes were being read, the copy is a splice, so it is dropped and the previous frame stands.
3. **An environment switch, `DX11_NO_DAMAGE`,** restores the old behaviour, so both can be measured inside one binary.

The second difference is visible in the source:

```
/* Painted while we were reading? Then the bytes we took splice two
 * guest states; keep the last whole frame and take it again. */
if (riscos_blitter_take_damage()) {
    fb_seen_damage = true;
    pending = true;
    if (fb_damage_gate() && fb.have_good && ++held < FB_SETTLE_MAX) {
        fb_dropped++;
        return;
    }
    held = 0;
}

fb.raw_cur = next;
fb.have_good = true;
fb_uploads++;
```

What it saved

Host	Measurement	Result
Mac	frames copied, 4,500 frames with a window opening, scrolling and typing	25.0%
Windows	1920 × 1080 idle, gated	2,676 copied, 1,223 skipped, 1,309 ms copying
Windows	the same, ungated	4,199 copied, 0 skipped, 2,141 ms copying

The Windows commit states the honest conclusion. A full 8 MB copy costs 0.49 ms, so copying every frame at 60 Hz costs about 3% of one core, and saving a third of that saves about 1%. The change is still right: it is less work, it matches the Mac's design, and it retired a worse mechanism. But it is not where the guest's time goes.

The blind spot

INFERRED, from the code: only the blitter raises the damage word. Two kinds of drawing never do:

- **window drags and scrolls**, which go through the video driver's DMA copies
- **text and lines**, which the kernel plots straight into memory

With GVFill loaded, that drawing reaches the screen only on the four-frame timeout. At 60 Hz that is every fourth presented frame, an effective 15 Hz. It matches the 25% copy rate the Mac measured, and the Metal commit names text and lines explicitly. The pointer is unaffected, because it is composited every frame.

Covering those paths would close the gap. The DMA model could raise the same word after a 2D copy. **INFERRED:** that is a small change, but nobody has made it.

The real fix: screen banks

Every scheme so far has had to guess when the guest has finished drawing, and a wrong guess shows a torn frame. The fork's screen-bank design, `BANKS.md`, aims at making a wrong guess harmless instead. The idea came out of a session watching the desktop tear. The design quotes it in one line:

```
create fb, display fb, copy fb -> bb, update bb, swap
```

RISC OS already has the pieces. The kernel sizes its screen memory for two banks, and there are calls to choose which bank is displayed and which is drawn into. The Wimp draws through `OS_Plot` and `OS_SpriteOp` and never touches the screen address directly. The video driver can pan. A live framebuffer at 1920 × 1200 is allocated at 1920 × 2400: room for a second bank.

The prize is not speed. **A mistimed swap shows a complete older frame instead of a half-drawn one** — late rather than torn.

Why the copy is the part that matters

The Wimp redraws *incrementally*. When something changes, it gives each task a list of damaged rectangles and repaints only those. Flip a bank under that, and the back buffer holds the frame from two flips ago: the backdrop, other windows and the icon bar would all snap back to stale content. Copying front to back first makes the back buffer correct before anything draws into it.

That copy is also why RISC OS does not double-buffer on real hardware: on a Pi it is 9.2 MB of block memory moves every frame. On the host it is one `BLIT_OP_COPY`, which the blitter implements and has never been asked to perform.

Per-bank damage, and why it belongs in the Wimp

The design's sharpest argument is about where the fix finally belongs. With two banks, **a damaged rectangle is not correct until both banks have been repainted**. Damage stops being a per-frame list and becomes per-bank: each rectangle has to survive until every bank has seen it. The damage lists live inside the Wimp, and nothing outside it can see them. So:

- **A module can prove the idea.** It hooks entry to `Wimp_Poll`, when a task has finished its work, then swaps banks, copies the whole screen front to back, and points VDU output at the other bank.
- **A Wimp that knows it is double-buffered needs no copy at all.** It redraws the union of this frame's damage and last frame's into the back buffer. That turns a fixed full-screen cost per frame into a cost proportional to what changed, which on an idle desktop is nothing.

The design notes this would be worth having upstream: the desktop tears on real hardware too, and the copy's cost is the only reason RISC OS does not double-buffer.

The plan, and its status

Step	What
1	the front end follows the displayed bank instead of the framebuffer base; a no-op while nothing banks
2	a separate module, not GVFill, so display sync and blitting do not share a fate
3	drive it from the Wimp_Poll pre-filter: swap, copy with BLIT_OP_COPY, retarget VDU output
4	measure: count swaps and copy cost from the blitter trace, and judge tearing by eye

None of this is built. The design lists its open questions first, and insists the measurements come before code, because every wrong turn so far came from writing code first. Two gaps are visible from reading the code (**INFERRED**):

- GVFill's fills use framebuffer-relative offsets, so drawing into a second *write* bank would need it to follow the screen start address.
- QEMU's framebuffer model caps the virtual height at 2,560 rows, so two banks taller than 1,280 rows would be clamped. That matters for the 4K modes a later sprint plans.

8. COMPLETE is the clock

RISC OS on the emulated Pi 4 was silent for its first day. The sound design that changed that opens with a finding that shaped everything after it: **the sound path is a conversation, not a chip**. This chapter follows that conversation: why accepting it once hung the machine, the one message whose rate is RISC OS's audio clock, two protocol bugs that only sound could expose, and why Windows stayed silent for two days after the Mac had sound.

There is nothing to emulate

On a Raspberry Pi 4, RISC OS's `BCMSound` module does not drive an audio chip. It is a client of a VideoCore service. It opens VCHIQ service `'AUDS'`, and ships 16-bit PCM to the GPU over it. No PWM, I2S or codec is reachable.

QEMU, for its part, modelled no Raspberry Pi audio at all. The I2S block is an unimplemented-device stub, and `hw/audio` contains nothing from Broadcom. So there was nothing to fix and nothing to build on. The design's summary: sound is **nine message types and one bulk transfer**, on a channel the fork already owned, handed to QEMU's audio back end — CoreAudio on the Mac, DirectSound on Windows, both already compiled in.

Why accepting `'AUDS'` once hung the machine

The VCHIQ peer had refused `'AUDS'`, because accepting it walked BCMSound into a blocking wait. The sound design traced exactly where.

The three requests that wait

BCMSound has a send-and-wait routine for requests that need an answer. It spins on a result flag with interrupts enabled and **no timeout**. Three requests use it:

Request	When BCMSound sends it	Needs
CLOSE	a sample-rate change; module shutdown	a RESULT
CONFIG	every sample-rate change	a RESULT
CONTROL	a mixer change	a RESULT

Initialisation sends only `OPEN`, and does not wait. So accepting the service costs nothing at boot. The hang comes only when something touches the sample rate. That is why sprint S1 was worth doing even if nothing had followed: it removed a hang from a path RISC OS takes whenever anything touches the rate.

The whole conversation

A message is a 32-bit type plus 16 bytes of payload, 20 bytes in all:

Direction	Message	Payload	Reply
to the GPU	OPEN, CLOSE	none	RESULT for CLOSE
to the GPU	CONFIG	channels, sample rate, bits	RESULT
to the GPU	CONTROL	volume, destination	RESULT
to the GPU	START, STOP	draining flag for STOP	none
to the GPU	WRITE	count, two cookies, silence, max packet	COMPLETE, repeatedly
from the GPU	RESULT; COMPLETE	success; count and cookies	—

The protocol matches the definitions in Linux's Pi audio driver, and BCMSound agrees with them message for message. `CONFIG` always asks for **two channels at 16 bits**, at one of nine rates from 8,000 to 48,000 Hz. That maps exactly onto QEMU's audio settings: signed 16-bit, little-endian, stereo. BCMSound sends zero cookies and ignores them; Linux sends and checks its own. The peer echoes whatever arrives, which keeps both clients happy.

COMPLETE is the clock

This is the part of the design that makes it cheap, and it follows from how BCMSound counts. Paraphrasing its callback: each `COMPLETE` reports a number of bytes consumed, with bit 30 as an *audio starved* flag. BCMSound adds the counts up. Each time the total crosses one buffer's worth, it tells the sound system a buffer completed, and the sound system fills the next one.

So **the rate at which COMPLETE arrives is the rate at which RISC OS generates sound**. There is no other clock in the loop.

That means the pacing should come from the audio device itself. QEMU's output API is a *pull* model. The back end calls a device callback with the number of bytes it can accept, at the real rate the host's sound card consumes. Report exactly what the back end took, and the guest is clocked by the actual output device:

```
/*
 * The sound card's pull. Whatever it takes out of the ring is what the
 * guest is told has played, so the reports come out at the rate the host
 * actually consumes audio -- no timer to tune and nothing to drift
 * against. Called from the audio timer on the main loop, BQL held, which
 * is what makes it safe to queue a message and ring the doorbell here.
 */
static void auds_audio_cb(void *opaque, int avail)
{
    BCM2835VchIQState *s = opaque;
    uint32_t reported = 0;

    if (!s->auds_open || !s->voice) {
        return;
    }
    while (avail > 0 && s->ring_used) {
        uint32_t run = MIN((uint32_t)avail, s->ring_used);
        size_t took;

        /* One contiguous piece at a time: the ring wraps, the API does not */
        run = MIN(run, s->ring_size - s->ring_tail);
        took = audio_be_write(s->audio_be, s->voice, s->ring + s->ring_tail,
                               run);

        if (!took) {
            break;
        }
        s->ring_tail = (s->ring_tail + took) % s->ring_size;
        s->ring_used -= took;
        reported += took;
        avail -= took;
    }
    if (reported) {
        auds_complete(s, reported);
        vchiq_signal_guest(s, s->slave_base + VCHIQ_SS_TRIGGER);
    }
}
```

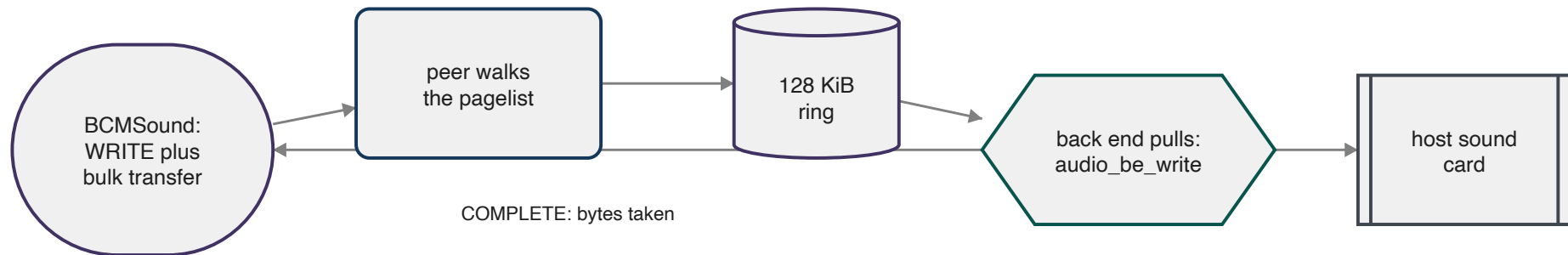
```
}
```

One ring buffer, 128 KiB, sits between the two sides. The VCHIQ side fills it from bulk transfers, the callback drains it into the sound card, and the bytes drained become the next `COMPLETE`. Both ends run under QEMU's big lock, which is where the peer already queues messages and rings the doorbell.

If the ring is ever full, the oldest audio is dropped, not the newest:

```
if (len > space) {
    /*
     * Only reachable if the host stopped consuming: the guest sends
     * more only when we report, so it cannot outrun us by itself.
     * Drop the oldest rather than the newest, so what plays next is
     * what the guest sent most recently.
     */
    uint32_t drop = len - space;
    ...
    s->ring_tail = (s->ring_tail + drop) % s->ring_size;
    s->ring_used -= drop;
}
```

The audio loop



Priming matters too. On `START`, BCMSound sends a burst of buffers up front, about 50 milliseconds' worth, and the ring must be able to take the whole burst before the first `COMPLETE` goes back.

Bug 1: the stall at exactly 1,265 buffers

The first build, with the service accepted and the loop paced, stalled after exactly **1,265 buffers**. It did so in three runs, headless and windowed alike, so it was not a race. There was no error and no `STOP` from the guest. The peer's last act each time was a `COMPLETE` the guest never answered.

The cause was in VCHIQ itself, not in sound. VCHIQ's slot protocol is symmetrical. Each side hands the other's used message slots back: it appends the slot index to the owner's queue, bumps a recycle counter, and fires a recycle event. The guest did that faithfully for the peer's slots. **The peer had never done it for the guest's.** It had never mattered, because before sound the peer received a `CONNECT` and four `OPEN`s in its whole life, comfortably inside one 4 KB slot. Sound sends 48 bytes per buffer. After about fifteen slots the guest ran out, and blocked with nothing to say.

The fix is nine lines:

```
static void vchiq_recycle_slot(BCM2835VchIQState *s, uint32_t slot)
{
```

```
uint32_t n = vchiq_ld(s, s->slave_base + VCHIQ_SS_SLOT_QUEUE_RECYCLE);

vchiq_st(s, s->slave_base + VCHIQ_SS_SLOT_QUEUE + (n % s->per_side) * 4,
        slot);
vchiq_st(s, s->slave_base + VCHIQ_SS_SLOT_QUEUE_RECYCLE, n + 1);
trace_bcm2835_vchiq_recycle(slot, n + 1);
vchiq_signal_guest(s, s->slave_base + VCHIQ_SS_RECYCLE);
}
```

Afterwards the loop ran indefinitely: 5,883 buffers in a 100-second run. The sound design's comment on it: a bug that sits harmless in a peer that barely talks, and becomes a hard stop the moment one does.

Bug 2: silence that looked like working code

Sprint S2 read the actual samples, and wrote them to a WAV file to be looked at. A bulk transfer does not carry the buffer's address. It carries the address of a *pagelist*: a length, a type, an offset, then a list of entries, each a run of physical pages. Built to Linux's documented packing, the WAV file was **digital silence** while the guest was demonstrably playing.

The trace showed two pagelists alternating, as a double buffer should, and both resolving to the same address. Dumping the raw structure instead of the interpretation showed entries of `0x00201a00` and `0x00201b00`. Those were 256 bytes apart, which cannot be two 2 KB buffers, and not page-aligned, which the Linux encoding requires. Masking with `~0xfff` had rounded both down to a page of zeros.

RISC OS's own VCHIQ port explains it. Paraphrasing: the entry encoding depends on the width of a physical address. With 32-bit addresses, the top 20 bits are the address and the low 12 are a page count. **With 36-bit addresses, the address is shifted right by four, and only the low 8 bits count pages.** A BCM2711 is a 36-bit part. Decoded that way, the two buffers come out at `0x201a000` and `0x201b000`, one page apart:

```
entry = vchiq_ld(s, pagelist + VCHIQ_PAGELIST_ADDRS + entry_index * 4);
base = (uint64_t)(entry & ~VCHIQ_PL36_COUNT_MASK)
    << VCHIQ_PL36_ADDR_SHIFT;
pages = (entry & VCHIQ_PL36_COUNT_MASK) + 1;
```

The design draws the lesson in one sentence: Linux's driver is the specification for the *protocol*, but RISC OS's port is the specification for what RISC OS puts on the wire, and where they differ, only one of them is the guest.

The clock, measured

Sprint S1 had paced COMPLETE from the fork's virtual clock, because there was no audio device to ask yet. Sprint S3 deleted that clock:

Build	Audio delivered	Of real time
S1: a timer deadline per buffer	73.4 s of 96 s	0.76×
S1: one virtual clock, reporting what has played	68.3 s of 96 s	0.71×
S3: the sound card is the clock	29.95 s of 30.0 s	0.998×

The design is precise about why S3 works. It is not that the new code is more careful: there is no longer a model of how fast audio should play. The host's sound card consumes at the real rate *because it is* the real rate. Reporting its consumption verbatim makes the guest's clock the same clock, with nothing to tune and nothing to drift.

The measurement needed care of its own. The first two attempts divided the audio delivered over a whole run by a *guessed* start time, and gave 0.65× and 0.68×. That sent the investigation looking for a starvation bug that did not exist: the ring sat steady at 10,240 bytes throughout. Sampling over a known thirty-second interval gave 0.998×. The ratio was always right; the denominator was a guess.

With no audio back end at all, the old virtual-clock pacer takes over again, so the guest's sound loop still turns. Silence that still boots is not an error.

Why Windows was silent for two days

Sound landed on 10 September and played on the Mac. On Windows it was reported as not working, and the protocol was not the problem. Commit e61d3314bd found two causes.

Nothing on Windows asked for audio. Sound needs two command-line options: a back end, and the global property that hands it to the VCHIQ peer. The Mac launch scripts passed both from the first day. The Windows launcher and the test farm passed neither. Without them nothing fails and nothing warns.

The service is accepted, the loop turns, and **silence is the only symptom** — which is also the symptom of sound never having been built.

DirectSound then killed QEMU outright. QEMU's DirectSound back end initialised the *capture* device unconditionally, and treated failure as fatal. On a desktop with speakers and no microphone, the whole audio device failed and QEMU exited. Recording is now optional:

```
if (SUCCEEDED(hr)) {
    hr = IDirectSoundCapture_Initialize (s->dsound_capture, NULL);
}
if (FAILED(hr)) {
    if (s->dsound_capture) {
        IDirectSoundCapture_Release(s->dsound_capture);
        s->dsound_capture = NULL;
    }
    warn_report("dsound: no recording device (%lx); playback still works,"
               " capture is unavailable", (unsigned long)hr);
}
```

The proof did not rely on listening. With the WAV audio back end over a 91-second boot, peak amplitude was **16,302 of 32,767**, with signal at 1.0 s and 1.5 s for the boot beeps and from 58.5 to 60.0 s for BASIC `SOUND` notes. A silent capture is a file of zeros of the same size, so a growing file is not the test. The peak is.

What is left

Item	State
CONTROL volume and destination	acknowledged, not applied; use the host's volume
a mid-stream rate change	reopens the voice and drops the ring; RISC OS sets the rate once in practice
the starved bit	never set; a dry ring reports a smaller count instead
capture	not implemented
the ring across a snapshot	not migrated

INFERRED — latency was never measured by the project. A 2,048-byte buffer at 44.1 kHz stereo is about 11.6 ms. The six-buffer priming burst is about 70 ms, and a steady ring of 10,240 bytes is about 58 ms. Add the back end's own buffering and the audio timer's 10 ms period.

9. Measured, and open

This last chapter gathers the numbers from the whole document in one place, each with its source. It then lists what is still open: first as the project states it, then as found by reading the code. It ends with the places where the fork's own documents have fallen behind its code, so a reader knows which to trust.

The numbers

Display

Measure	Result	Source
Windows decode against the guest framebuffer	84 of 480,000 pixels differ: the clock	418847ac82
Intel Mac decode against a QMP screendump	134 of 480,000 differ: the pointer, mid-move	cf4a309fcd
cost of the window to the guest	about 1,580 → 1,540 M/s, -2.3%, within noise	sprint U4 record
rows changing between frames at rest	0.1%	6c26b418f0
Mac frames copied with the settle rule	25.0% of 4,500	6c26b418f0
Windows, gated against ungated, 1920 × 1080 idle	1,309 ms against 2,141 ms of copying	b3d60dbe6a

Pointer and scripting

Measure	Result	Source
pointer traffic in one boot from the card	69 shape updates, about 356 move transactions	graphics design §8a
opaque texels in the arrow	134 of 32 × 32	graphics design §8a
Apple Event round trip against QMP	16.7 ms against 0.1 ms	scripting design, Q2
screendump against screenshot	40 ms against 72 ms	scripting design

The blitter

Measure	Result	Source
sprite plot, guest against host	490 μs against 80 μs: 6.1×	a3f6b6fafa
sprite pixels on the host	94.3%	8b1d43fb4b
correctness	0 of 2,304,000 pixels differ	a3f6b6fafa

The blitter, continued

Measure	Result	Source
full-screen 9 MB fill, memset against synchronous Metal	68 μ s against 510 μ s	tools/fillbench.m
one contiguous 9 MB write against row by row	68 μ s against 118 μ s	riscos_blitter.c comment
8 bpp title-bar fill; full-screen erase	2 μ s; 303 μ s	0223e6f2d8
ROM-spliced module, one boot	26 fills at 1–2 μ s each	c0a55e4f44

Sound

Measure	Result	Source
delivery, the sound card as clock	29.95 s of 30.0 s: 0.998x	sound design §12
delivery, virtual clock (superseded)	0.76x and 0.71x	sound design §10
buffers before the slot fix	1,265, then a stall	sound design §10
buffers after the slot fix	5,883 in 100 s	sound design §10
WAV capture of a 91 s boot	peak 16,302 of 32,767	e61d3314bd

Open, as the project states it

- **GVFill declines masked sprites** and any plot with plot-action bit 4 set. The meaning of bit 4 is not established.
- **Packed sprite sources** through a colour table exist on the host, and nothing reaches them.
- **Cross-depth plots**, which include every desktop sprite onto an 8 bpp screen, are passed to SpriteExtend by design.
- **The Mac's settle-copy saving** is capped at 75%, because text and lines never raise the damage flag.

- **Fill in the project's own ROM** remains the long-term design. The asynchronous GPU blit is gated on a measurement never taken.
- **In the ROM, GVFill never sees a sprite plot**, so it is loaded from the share instead.
- **Sound**: volume is acknowledged but not applied; there is no capture; the ring is not migrated.
- **The screen-bank design** is unbuilt.
- **Mac pointer grab, full screen and the snapshot menu** are unexercised.
- **Sprint 13's wall-time targets** — a Pinboard drag, a NetSurf scroll — have no recorded result. The redraw benchmark tool exists, but no run of it is recorded.

Open, found by reading the code

Each item below is **INFERRED**: read from the code, not tested.

Where	What
bcm2835_fb_get_config	the seqlock reader's continue inside a do ... while jumps to the loop test; if a write is in flight and the generation has not moved, it returns without filling *out
bcm2835_vchiq_get_cursor	staleness is checked before the caller copies the 64 × 64 image, so a commit in between could tear the sprite for a frame
vmstate_bcm2835_vchiq	lists disp_res_w and disp_res_h twice
the damage word	DMA copies and kernel text and lines never raise it, so that drawing updates at the 4-frame timeout: 15 Hz at 60 Hz

Found by reading the code, continued

Where	What
the Metal front end	gates on the damage flag even when GVFill is not loaded; the Windows twin guards against that
riscos_blitter.c	the host copy and the mapped physical sprite source are never reached by shipped guest code
bcm2835_dma.c	a comment says RISC OS uses DMA pattern fills for FillRectangle, while the graphics design's reading of the sources says nothing uses that mode
the blitter on an Intel Mac	shared Metal buffers imply a bus transfer on a discrete GPU; the frame cost there is unmeasured

Where the documents have fallen behind

The fork's documents are unusually thorough, but they were written sprint by sprint. Where a document and the code disagree, the code and the later commit messages are right:

Document	Says	The code, since
both blitter READMEs	98.2% of sprite pixels	94.3%, since the masked-sprite backout 8b1d43fb4b
emulator README	the Windows front end paces to vsync and fingerprints rows	both removed, c5f7bb86b7 and b3d60dbe6a
macOS record	Direct3D reads a texture; middle-click grabs	a byte buffer since ec4ace18b4; all buttons are the guest's since 295ea58e4d

Where the documents have fallen behind, continued

Document	Says	The code, since
ui/metal.m comment	middle click captures the pointer	as above
UI design	raw-input relative mouse	an absolute tablet since a59d153d17

Where the documents have fallen behind, concluded

Document	Says	The code, since
riscos_blitter.h, .c	sprite rows are stored bottom first	the path in use runs top down, 52550e8d9a
riscos_blitter.c fill comment	the Metal front end copies every frame	the settle rule, 6c26b418f0
GVFill header comment	the RMA is identity mapped; storage lives in the module body	neither: the device header says otherwise, and 1.03 uses an RMA workspace
qapi/ui.json	vsync pulses follow a presented frame	the generator is independent of rendering
the design record's render table	fill parameters as left, bottom, right, top	GVFill reads left, top, right, bottom, and draws correctly

The public-facing site repository also still says Windows has no sound, and gives the old middle-click grab and a `Shift+F10` Menu key that the Windows front end does not handle.

A note on provenance

The design notes behind this document quote their sources closely. That is how they stay checkable. It also means several public notes carry passages of RISC OS Open's own text: BCMSound's description of itself and parts of its assembly in the sound design, a kernel comment in the screen-bank design, and a video-driver comment in the UI design. The emulator README states that no RISC OS Open material is included, and the graphics design states that

nothing of the sources is copied into the tree. Those passages contradict both statements. This document paraphrases every such passage, and quotes only the project's own words.